ARQUITECTURA DE AGENTES EDUCATIVOS • CLOUD NATIVE TRIFECTA

Tutor IA Contextual con MCP y GraphRAG

Cómo la interoperabilidad modular y la recuperación híbrida eliminan la obsolescencia técnica y garantizan soberanía absoluta.

El Desafío de la Ceguera de Versiones

0%

Tolerancia a APIs deprecadas en entornos de ingeniería cloud native.

Los LLM genéricos mezclan versiones de software pasadas y carecen de conexión directa al estado real de tu infraestructura o especificaciones oficiales.

02 / ARQUITECTURA DE TRES CAPAS

La Cloud Native Trifecta & Protocolo MCP

CLIENTE / IDE: Open WebUI / Claude Desktop / Cursor

study-cybercirujas

El Syllabus & Labs: Objetivos de examen y rutas prácticas.

• 38 Certificaciones • Break/Fix • Zero Tokens

k8s-docs-mcp

La Verdad API & YAML: Documentación oficial sincronizada.

• OpenAPI v3 • Manifiestos Canónicos • Sync

rag-tutor

La Arquitectura: Libros técnicos y GraphRAG avanzado.

• KùzuDB • Cross-Encoder • Agentic Loop

MOTOR OLLAMA LOCAL: qwen2.5:7b / bge-m3

100% Soberano & Offline (GPU)

03 / EXPERIENCIA CONTEXTUAL EN VIVO

Un Bot que Sabe lo que Estás Haciendo

01. Contexto Local

Terminal & Entorno

El agente detecta el archivo que editas, el clúster activo o el error en consola.

02. Orquestación MCP

Consulta Simultánea

Cruza syllabus, specs oficiales de Kubernetes y patrones de arquitectura en segundos.

03. Guía Socrática

Validación Real

Responde con evidencia exacta y pistas orientadas al diagnóstico, no respuestas vacías.

04 / PRECISIÓN TÉCNICA RIGUROSA

Cómo Definimos y Logramos "Cero Alucinaciones"

Eliminamos la Ignorancia, no la Estocástica

Los LLM son probabilísticos por naturaleza, pero cuando se alimentan exclusivamente de fuentes deterministas, su margen de error desaparece.

Si una API de Kubernetes no existe en el OpenAPI Spec oficial o en la documentación inyectada vía MCP, el agente no la inventa: la bloquea.

Memoria Externa Determinista: El modelo actúa como motor de razonamiento sobre datos oficiales.

Trazabilidad Absoluta: Cada respuesta incluye la referencia al documento o versión exacta.

Actualización Automática: Los cambios upstream se reflejan al regenerar el índice.

Trifecta de Recuperación en rag-tutor

1. Léxica (BM25)



Precisión quirúrgica en flags, comandos y códigos.

2. Semántica (Vector)



Comprensión conceptual de consultas complejas.

3. GraphRAG (KùzuDB)



Navegación de dependencias y prerrequisitos técnicos.

06 / JUSTIFICACIÓN TÉCNICA

¿Qué Tecnología Usamos y Por Qué?

Componente	Tecnología	Justificación de Ingeniería
Protocolo Base	`Model Context Protocol`	Desacopla la lógica de los datos entre cualquier cliente y servidor.
Búsqueda & Rerank	`BM25 + Vectors + Cross-Encoder`	Garantiza precisión en sintaxis y comprensión semántica profunda.
Inferencia Local	`Ollama (Qwen2.5:7b / BGE-M3)`	Soberanía total, costo cero y cero filtración de datos a terceros.
Sincronización	`GitHub Actions + Sparse-Checkout`	Automatiza el fetch de upstream sin intervención manual.

Independencia Operativa y Licenciamiento

“Bajo licencia Apache 2.0 y una arquitectura local-first, los modelos y herramientas de IA son piezas intercambiables, asegurando que tu conocimiento e infraestructura permanezcan 100% bajo tu control.”

Cero vendor lock-in • Privacidad absoluta • Escalabilidad modular

ARQUITECTURA ABIERTA Y MODULAR

Tutoría IA Contextual con la Verdad de Hoy

Cloud Native Trifecta • MCP • GraphRAG • Ollama Local